



MARMARA
ÜNİVERSİTESİ

**MARMARA ÜNİVERSİTESİ TEKNOLOJİ
FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ
BÖLÜMÜ**

VERİ YAPILARI PROJE RAPORU

Proje Adı: Snake Game

Hazırlayanlar:

Kuzey Yağız Yıldız, 171424006

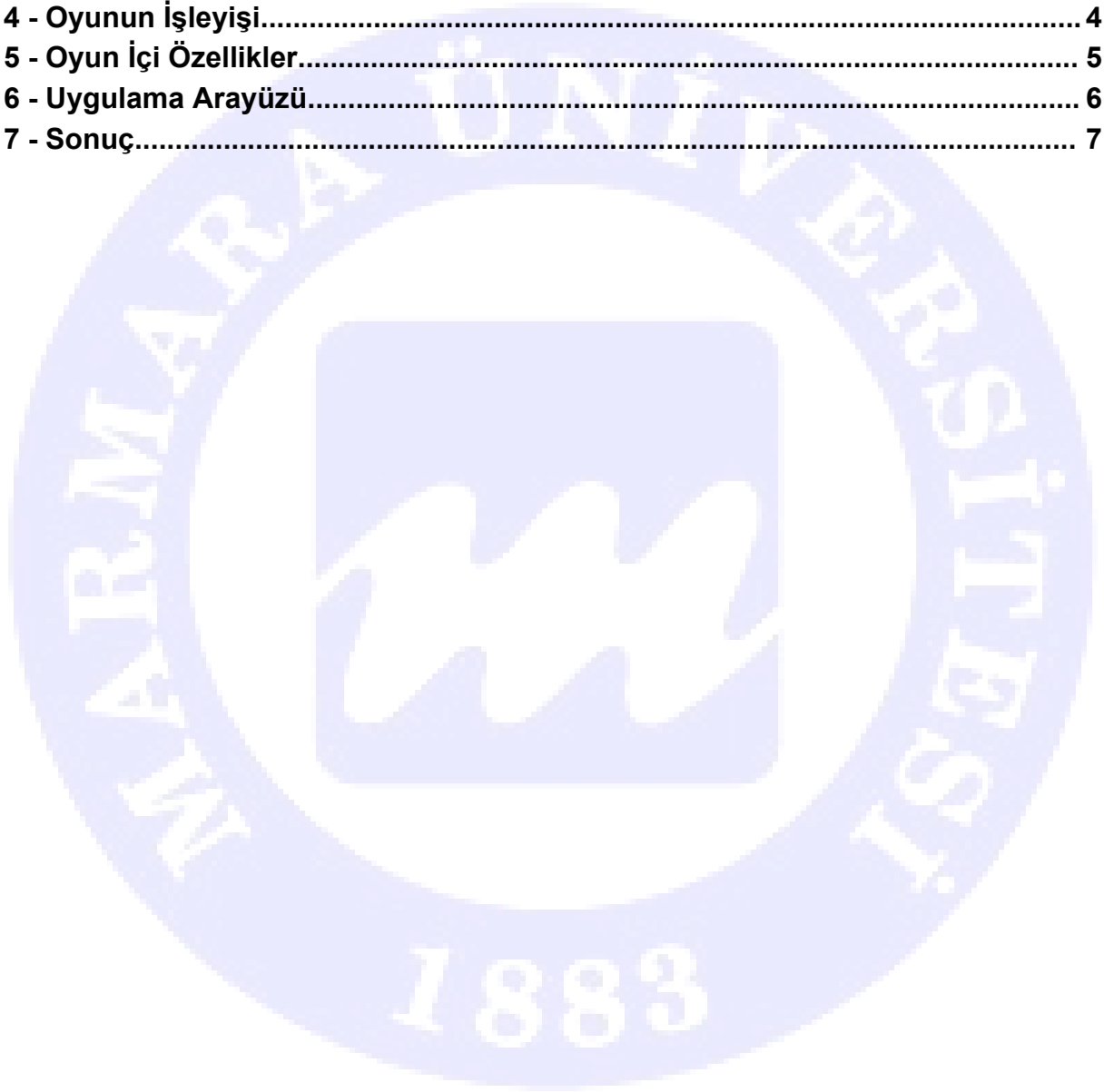
Ali Talha Yurtseven, 170424526

Oğuzhan Dünder, 170424004

Selin Müge Terzi, 170424026

İçindekiler

1 - Proje Özeti.....	2
2 - Dosya ve Veri Yapıları.....	3
3 - Sınıflar (Development View).....	4
4 - Oyunun İşleyişi.....	4
5 - Oyun İçi Özellikler.....	5
6 - Uygulama Arayüzü.....	6
7 - Sonuç.....	7



1 - Proje Özeti

Hashmap ve Bağlı listeler kullanılarak Java ile geliştirilmiş yılan oyunu.

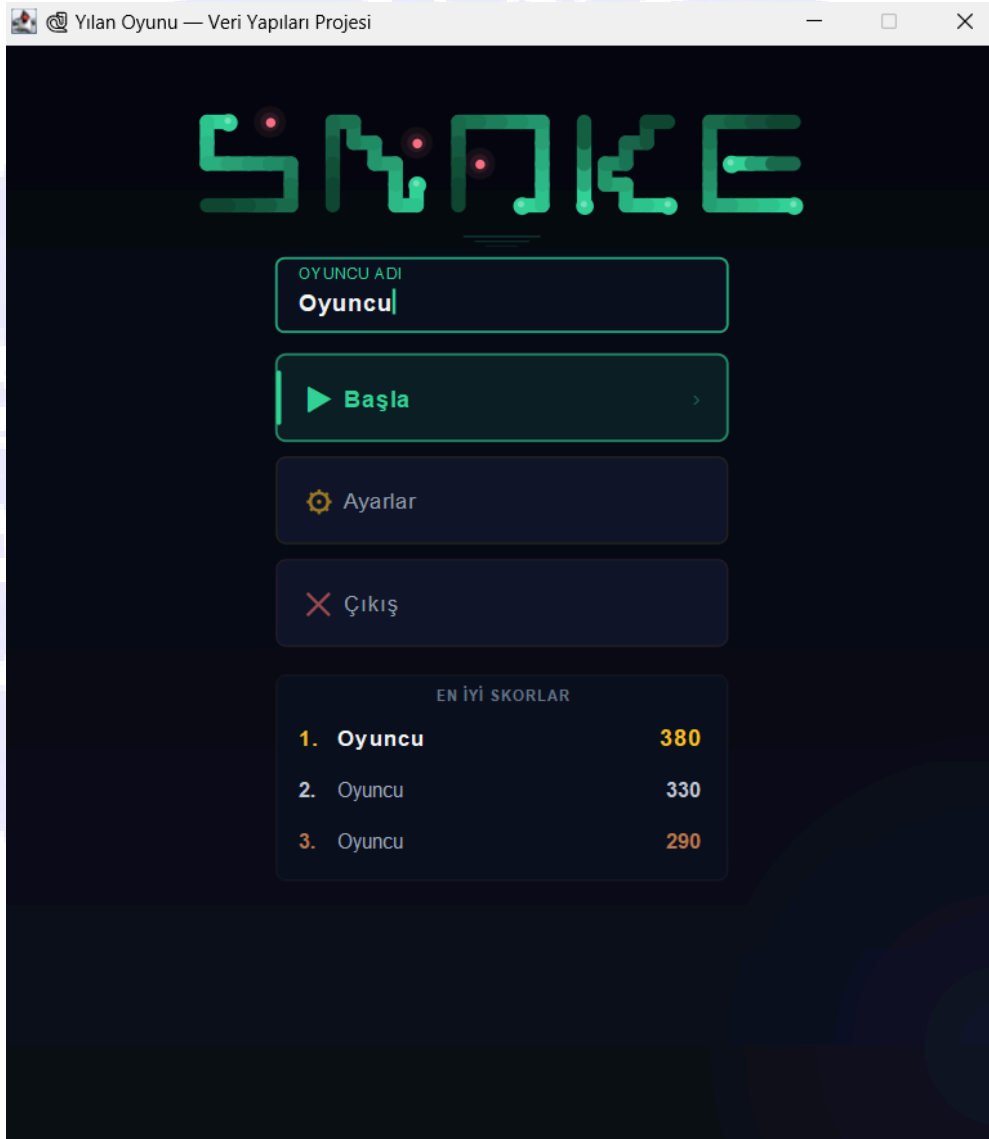
Oyuncu ismini, yılanın rengini, oyun modunu (wrap) değiştirebilir.

Yılan ekranda çıkan yiyecekleri yedikçe oyuncu puan alır.

Oyunda puan aldıkça belirli bölgelerde rastgele oluşan engeller, yılanın artan hızı ve boyuyla oyun zorlaşır.

Ekranda çıkan güçlendirmeler alınarak avantaj sağlanabilir.

En iyi üç skor ana ekranda gösterilir.



Görsel 1 - Snake Game Ana Ekranı

2 - Dosya ve Veri Yapıları:



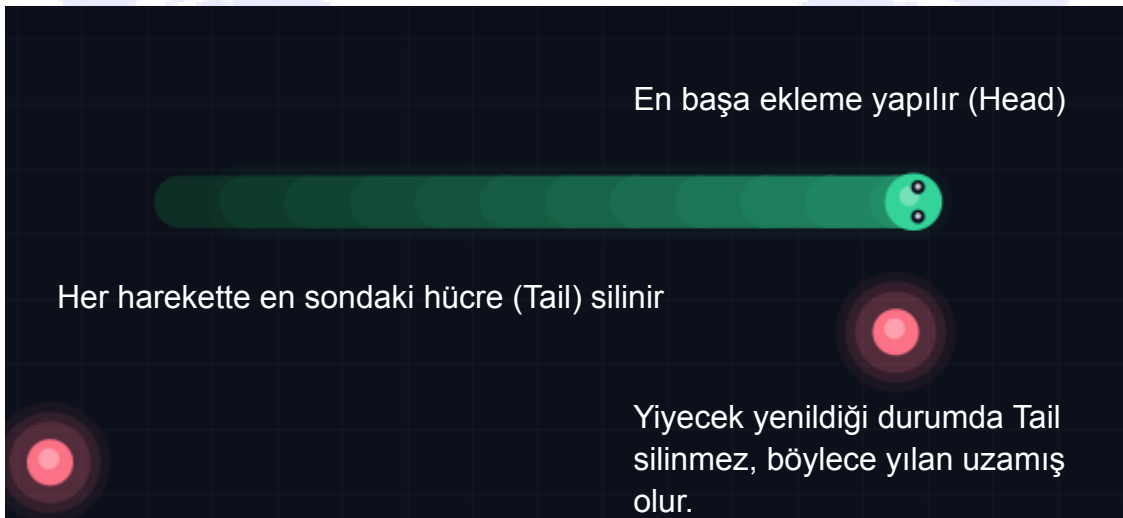
Görsel 2 - Dosya Yapısı

Java → Projenin yağımindaki kullanılan dil, ders kapsamı gereğince, Java'dır.

Java Swing → GUI (Graphic User Interface) için Java Swing kullanılmıştır.

Linked List → Yılanın yapısında linked list kullanılarak $O(1)$ zaman karmaşıklığı ile yılanın sadece Head'e ekleme yapıp Tail'den çıkartma yapılarak hareketi sağlandı. ArrayList kullanılsaydı zaman karmaşıklığı $O(n)$ olacağı için optimal olmazdı.

HashMap → Yılanın her hareketinde hücrede yemek olup olmadığı HashMap ile kontrol edilir, bu işlemin zaman karmaşıklığı $O(1)$ 'dir.



Görsel 3 - Yılan

3 - Sınıflar (Development View)

Nesne yönelimli programlama mantığı ile esnek ve katmanlı bir yapı oluşturuldu.

Main → GameWindow'u açarak oyunu başlatır.

Datastructures:

- **My... Sınıfları** → Hazır fonksiyonların kullanılması yerine özgün veri yapıları oluşturuldu.

UI:

- **GameWindow** → GamePanel'i açarak GUI'ı yapılandırır.
- **GamePanel** → Asıl oyuna dair GUI bileşenlerini tutan ve uygulayan yapı.

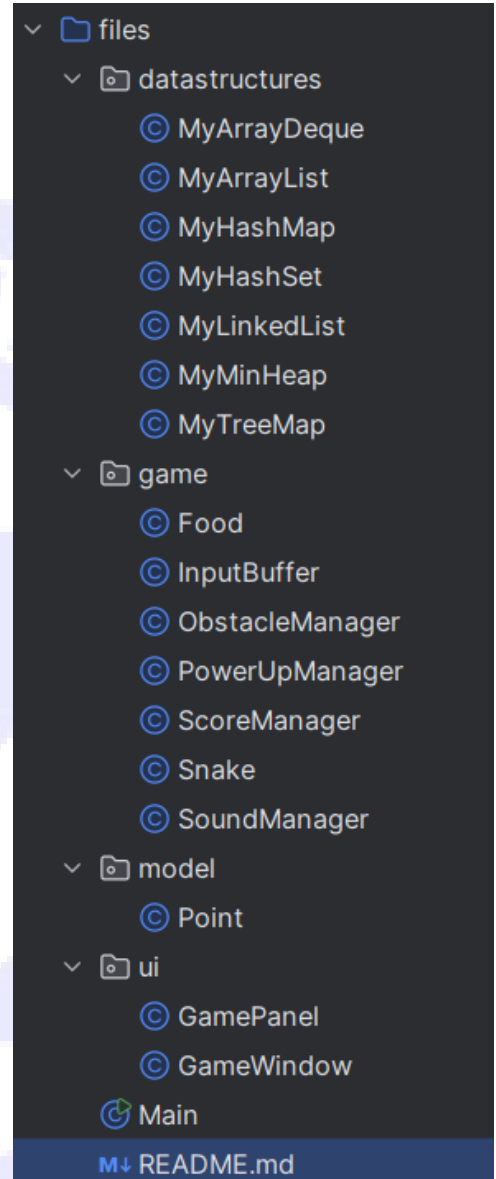
Game:

- **Food & Snake** → Oyun içindeki objeler.
- **...Manager Sınıfları** → Objelerle alakalı olayları ve değişimleri kontrol ederler.

Model:

- **Point** → Oyun tahtasındaki bir hücrenin koordinatını temsil eder. HashMap'te key olarak kullanılır.

Input Buffer → ArrayDeque ile basılan tuş kontrolü, FIFO mantığı.



Görsel 4 - Sınıflar

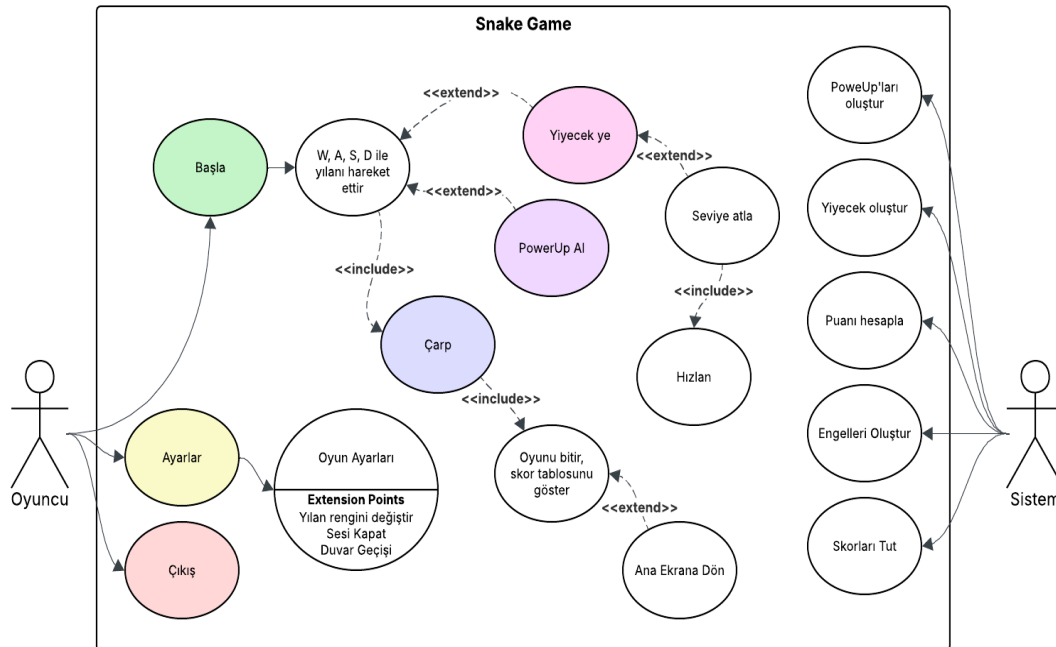
4 - Oyunun İşleyişi

UseCase Kategorileri:

- Başla: 8 use case
- Ayarlar: 2 use case
- Çıkış: 2 use case

Aktörer:

- Oyuncu: Primary Actor
- Sistem: Secondary Actor



GörSEL 5 - UseCase Diagram

5 - Oyun İçi Özellikler

Yiyecekler:

Yılan, spawn olan yiyecekleri yedikçe puan kazanır.

- Normal: +10 puan, en sık spawn olan.
- Bonus: +30 puan, spawn olma olasılığı %20.
- Süper: +50 puan, spawn olma olasılığı %10.

Hızlanma:

Yiyecek yiyerek puan kazandıkça yılan hızlanır.

Obstacles(Engeller):

Yılan seviye atladıkça (3. seviyeden itibaren) ekranın farklı hücrelerinde duvarlar spawn olmaya başlar. Oyuncunun bulunduğu yere fazla yakın spawn olmaması için bir güven mesafesi (safe radius) hesaplanır.

```
public class ObstacleManager { 7 usages

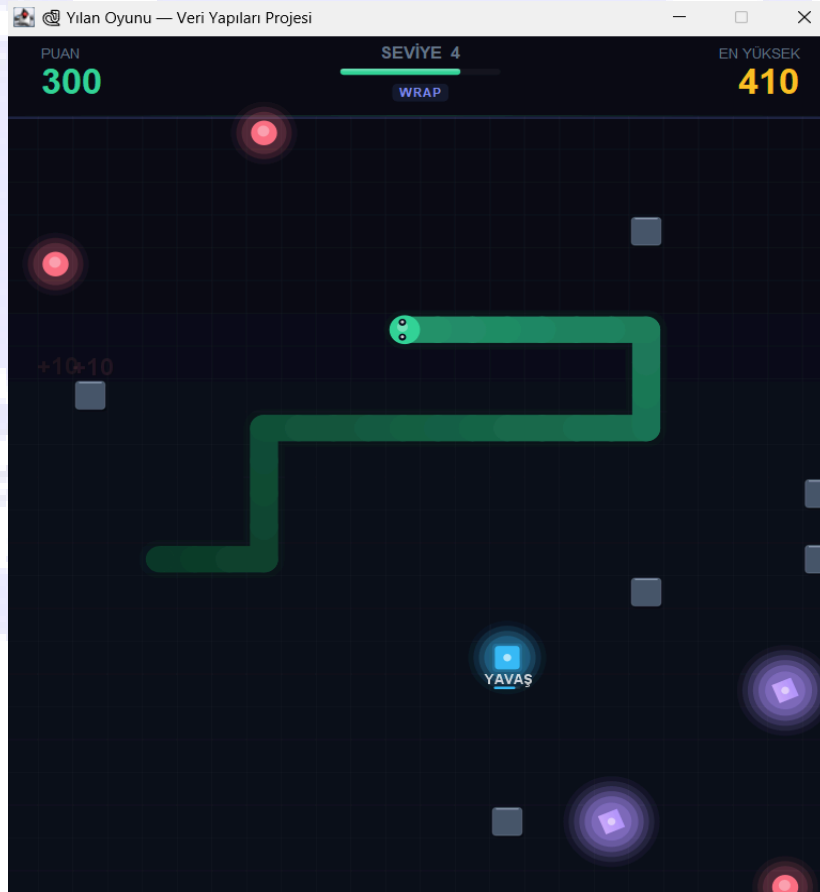
    private static final int ENGEL_PER_LEVEL = 3; 1 usage
    private static final int MIN_LEVEL = 3; 1 usage
    private static final int SAFE_RADIUS = 4; 1 usage
    private static final int ANIM_TICKS = 20; // kaç tick'te tamamlanır 2 usages
```

Görsel 6 - ObstacleManager

Power Up'lar:

- Slow: Timer'ı delay eder.
- Shield: Yılanı çarpma durumlarına karşı korur.
- Shrink: Yılanın uzunluğunu belli bir miktar kısaltır.
- Magnet: 2x puan sağlar.

6 - Uygulama Arayüzü



Görsel 7 - Oyun Ekranı

Wrap özelliği ana ekrandaki ayarlar kısmından veya oyun içinde “t” tuşuna basarak aktif edilebilir. Bu özellik sayesinde yılan, ekranın kenarındaki duvarlara çarpmak yerine ekranın diğer ucuna geçiş yapabilir.



Görsel 8 - Oyun Bitti Ekranı

Oyuncu bir engelle çarptığında bitiş ekranıyla birlikte kendi skorunu ve skor tablosundaki yerini görebilir.

7 - Sonuç

Bu projeyle birlikte veri yapıları ve algoritmalar dersi kapsamında öğrenilen farklı yapılar ve mantıksal işlemler pratiğe dökülerek pekiştirilmiştir.

Oyunun geliştirilme sürecinde sistemsel performansın önemi gereğince en optimal algoritma kullanılarak zaman karmaşıklığı olabildiğince indirgenmiştir.

Özetlemek gerekirse:

- LinkedList yazımı, uygulanması ve zaman karmaşıklığı ($O(1)$)
- HashMap yazımı, pratikte uygulanması ve zaman karmaşıklığı ($O(1)$)
- Java Swing ile GUI oluşturma ve oyun döngüsünü kurma
- OOP (NYP) ile kapsamlı bir proje geliştirme

GitHub linki: <https://github.com/ozzii-mrmr/Snake-Game>